GROUP 5 - FINAL PROJECT

DOCUMENTATION

==========

[Order System Management]

INTRODUCTION

Background:

This project presented to CCCS 105 - Information Management System 1 is a Order System that is designed to facilitate the Product Management and Ordering system for a business both for the user or admin and customer It allows users to create, read, update, delete as well as to maintain inventories of the product, upload images of the product and manage orders through a system that is developed using Flask and backed by a MySQL database.

Problem Statement:

Manual order processing is time-consuming, prone to errors, and difficult to track. This project aims to automate order management to improve accuracy, speed, and provide a centralized system for product and order information.

Scope:

The system covers product data management, including product details and images, order placement and tracking, and basic user authentication. It supports CRUD operations for products and orders but does not handle payment processing.

The system includes:

- Product management with image uploads
- Order creation and tracking
- User authentication with Admin and Customer roles
- Role-specific dashboards and access control
- Profile editing for customers
- CRUD operations for products, customers, and orders

Target Users:

1. Business administrators managing inventory and orders
2. ONLY Registered customers placing and reviewing orders

PROJECT OBJECTIVES
---

1. Develop a user-friendly GUI for managing products and orders
   - Provide forms to create, update, and delete product entries.
   - Display tabular views of products, customers, and orders with action buttons for easy navigation.
   - Responsive interfaces using Bootstrap for both admin and customer users.
   - Create separate dashboards for each role.

2. Integrate a MySQL database for persistent data storage
   - Design tables for products, orders, users, and order items

3. Develop an Order Management System Using Flask
   - Use Flask to build a modular, maintainable web application, Structure the application for scalability and feature expansion
   - Utilize Flask extensions such as Flask-Login for user authentication and session management, and Flask-SQLAlchemy for ORM-based database interaction.

4. Implement CRUD functionalities for products, customers, and orders
   - Customers can view, update, and delete their profiles.
   - Admins can perform full CRUD operations on products and customers.

5. Enable role-based authentication and authorization
   - Admin users can manage products, view all orders, and control customer data.
   - Customer users can register, log in, browse products, place orders, and manage their profiles.
   - Implement route protection with `@login_required` and role-based access control (Admin vs. Customer).

SYSTEM RULES
---

1. Creating a Product/ Management
   - Products must have unique id (primary key)
   - Uploaded images must be one of the allowed formats: PNG, JPG, JPEG, GIF
   - Maximum upload file size is 16MB
2. Order Processing

- Orders can only be created if the product is in stock
- Deleting a product cascades deletion to related orders
- Only logged-in **Customers** can create orders.
- Customers can only view their **own** orders, not others'.

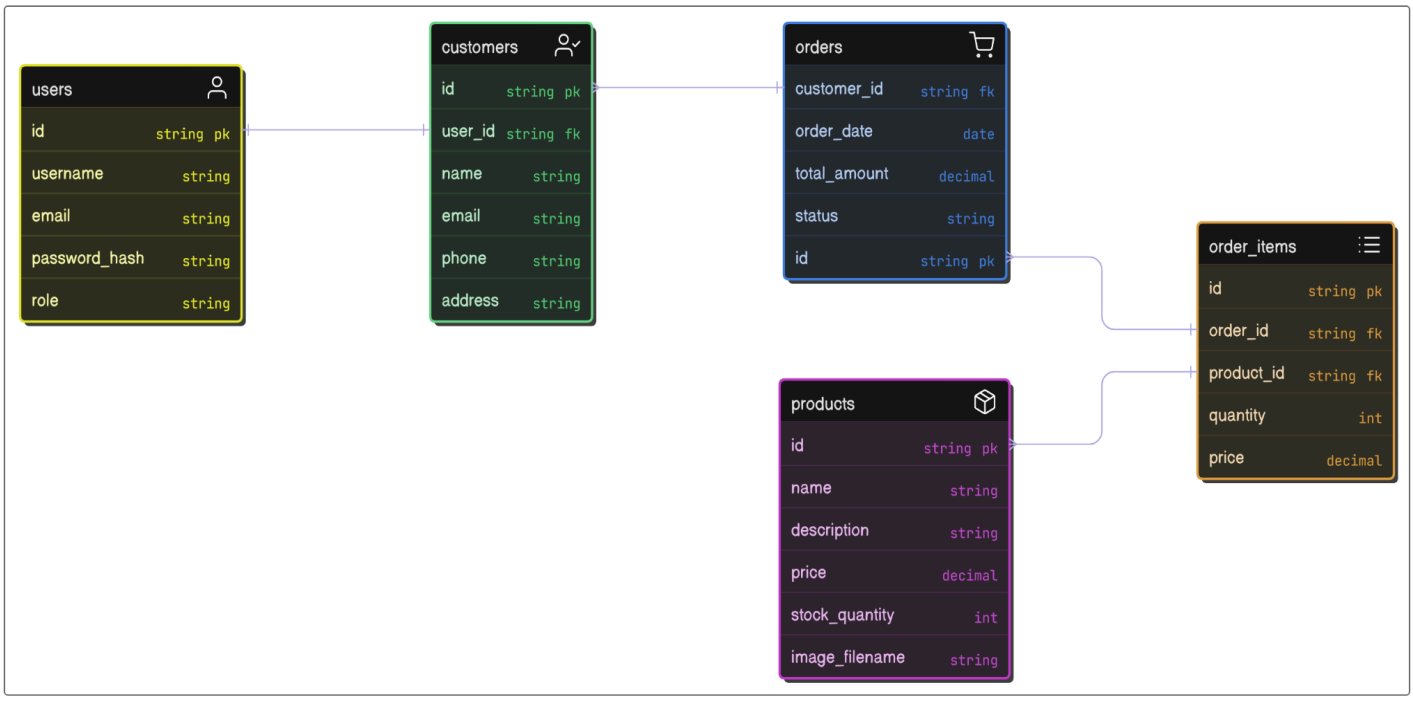
3. Role: User vs Admin
   - There are two roles: **Admin** and **Customer**.
   - Only **Admin users** are allowed to add, update, or delete products.
   - Only logged-in **Customers** can create orders.
   - Customers and Admin have separated Dashboard
   - Unauthorized actions or access attempts should be **blocked** and **redirected appropriately**

4. Authentication/Login and Session Rules
   - Users must log in to access features
   - Sessions maintain user roles and access rights securely
   - To test the functionalities from both the Admin and Customer perspectives simultaneously, use different web browsers or an incognito/private window to log in with separate accounts.

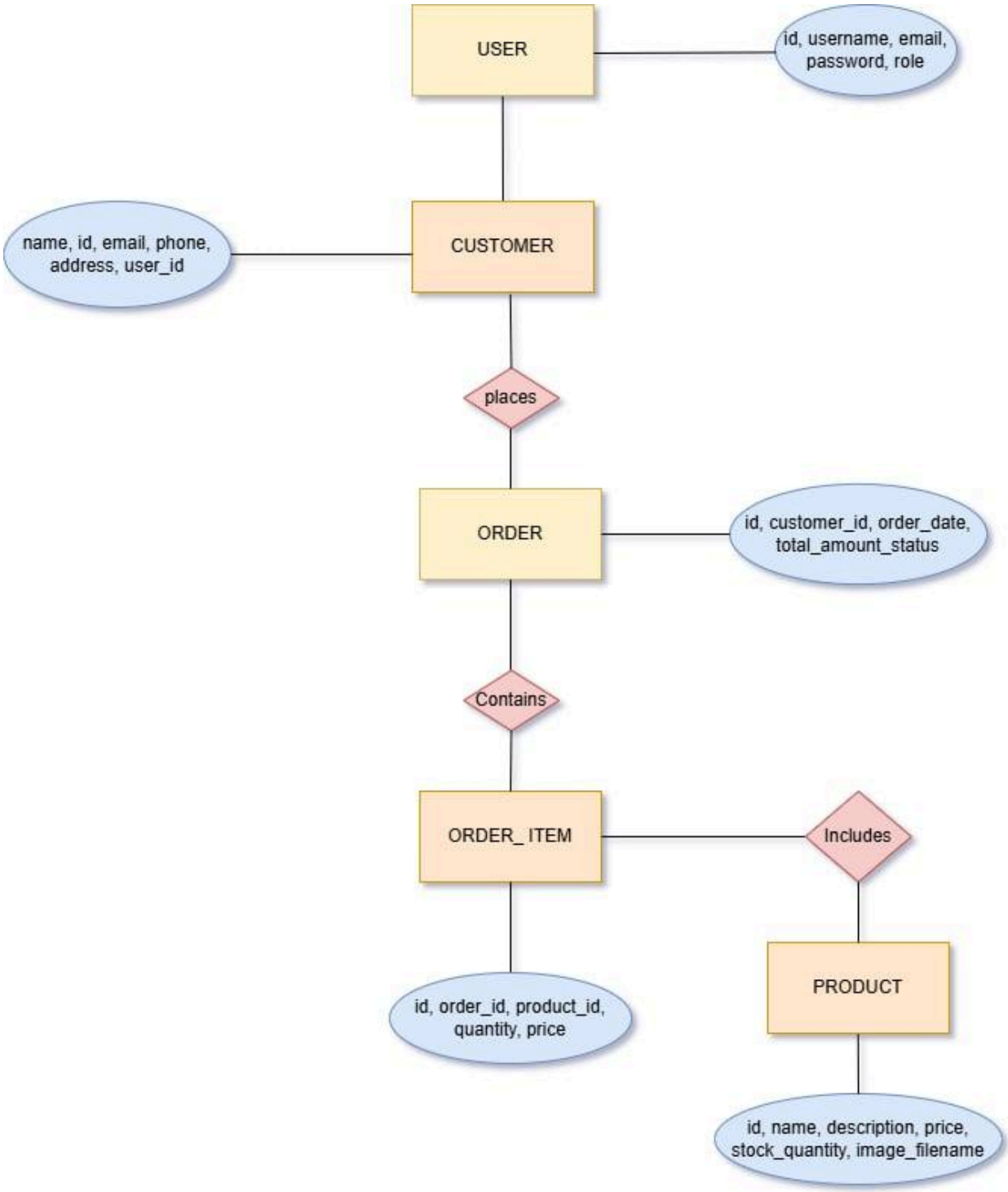
NOTE: Duplicate Data (excerpt to error login) (this applies also to duplicate products, customer name or customer login details)

DATABASE MODELS

RM:



ERD(CHEN NOTATION):



PROJECT OVERVIEW

This project is a web-based order management system built with Flask, SQLAlchemy, and MySQL. It supports both administrator and customer functionalities. Admin users can manage products and customers, including full CRUD operations and image uploads. Customers can register, update their profiles, browse products, place orders, and view their order history.

SETUP INSTRUCTIONS
---
1. Download the project repository, save it to the directory you want.

2. Open a cmd within the directory

3. Find the directory location of the file then Create and activate a Python virtual environment: ”python -m venv” –> venv\Scripts\activate

4. Install required dependencies: “ pip install -r requirements.txt”

5. Set environment variables:
   $env:FLASK_APP = "app.py"
   $env:FLASK_ENV = "init_db.py"

6. Initialize the database

7. Run the application: “flask run”

8. Open your browser at http://localhost:5000

Note: NEED TO LOGIN AS ADMIN FIRST after opening the website.
This is for Windows environment command

GROUP 5

- BACSAIN, Jave (**Lead Developer / Backend Specialist/Video**)
- COLICO, Carl Renz (***UI/UX Designer / Frontend Developer/Documentation/Video***)
- TERCERO, Michelle (**Database Designer / Tester/Initializer/Video**)

DEPENDENCIES

- Python 3.x

- Flask
- Flask-SQLAlchemy
- Flask-Migrate
- PyMySQL
- Other dependencies as listed in requirements.txt

RUNNING INSTRUCTIONS
---
1. Activate your virtual environment.
2. Ensure the MySQL server is running locally with the database cccs105 created.
3. Run the Flask application using: flask run in the same directory as your created virtual env.
4. Then, Access the GUI at http://127.0.0.1:5000 in your web browser.

DATABASE SETUP
---
1. Create MySQL database:
   CREATE DATABASE cccs105;

2. Configure database connection:
   - Database Name: cccs105
   - Username: root
   - Password: (no password)
   - Host: localhost
   - Port: 3306

3. Import schema and initial data:
   - Open MySQL Command Line Client
   - Use the following commands:
     mysql -u root CCCS105 < database/schema.sql
     mysql -u root CCCS105 < database/initial_data.sql

==========
Submission Instructions:
1. Create a Google Drive folder with your group name
2. Upload all project files following the folder structure
3. Set folder sharing to "Anyone with the link can view"
4. Submit the Google Drive folder link to the designated submission portal
==========